



Deep Reinforcement Learning

HOMEWORK 1 — VALUE-BASED METHODS

1 TD Learning with Linear Function Approximation

Question 1: TD-SGD Derivation

[2 Points]

Starting from the squared TD error loss

$$L_t(w) = \frac{1}{2} (R_t + \gamma \phi(S_{t+1})^\top w - \phi(S_t)^\top w)^2,$$

derive the stochastic gradient descent update for w and show that it has the form

$$w_{t+1} \leftarrow w_t + \alpha \delta_t \phi(S_t).$$

Answer:

To derive the update rule, we must use a **semi-gradient** approach. In Temporal Difference (TD) learning, the bootstrap target, $R_t + \gamma \phi(S_{t+1})^\top w_t$, is treated as a fixed constant with respect to the parameter w being updated. We do this because the target represents the goal we want our current value estimate to move toward, so we deliberately ignore its dependence on the weights during differentiation to ensure stable learning.

Let's denote the fixed target as $U_t = R_t + \gamma \phi(S_{t+1})^\top w_t$. The loss function can be rewritten as:

$$L_t(w) = \frac{1}{2} \left(U_t - \phi(S_t)^\top w \right)^2$$

Now, we take the gradient of the loss function $L_t(w)$ with respect to w . Using the chain rule and remembering that $\nabla_w U_t = 0$:

$$\begin{aligned} \nabla_w L_t(w) &= \left(U_t - \phi(S_t)^\top w \right) \nabla_w \left(U_t - \phi(S_t)^\top w \right) = \left(U_t - \phi(S_t)^\top w \right) (0 - \phi(S_t)) \\ \implies \nabla_w L_t(w) &= - \left(U_t - \phi(S_t)^\top w \right) \phi(S_t) \end{aligned}$$

Evaluating this gradient at the current weights $w = w_t$, and substituting U_t back into the equation:

$$\nabla_w L_t(w_t) = - \left(R_t + \gamma \phi(S_{t+1})^\top w_t - \phi(S_t)^\top w_t \right) \phi(S_t)$$

Notice that the term inside the parentheses is exactly the TD error definition:

$$\delta_t = R_t + \gamma \phi(S_{t+1})^\top w_t - \phi(S_t)^\top w_t$$

Therefore, the gradient simplifies to:

$$\nabla_w L_t(w_t) = -\delta_t \phi(S_t)$$

The standard stochastic gradient descent (SGD) update rule is:

$$w_{t+1} \leftarrow w_t - \alpha \nabla_w L_t(w_t)$$

Substituting our calculated gradient into the SGD rule gives the final update:

$$w_{t+1} \leftarrow w_t - \alpha (-\delta_t \phi(S_t)) \implies w_{t+1} \leftarrow w_t + \alpha \delta_t \phi(S_t)$$

Question 2: Update-Rule Comparison

[1 Point]

What are the similarities and differences between these two update rules?

Answer:

Similarities:

- **Core TD Mechanism:** Both methods are rooted in Temporal Difference learning. They update current estimates using the estimate of the next state before the episode terminates.
- **Bootstrapping:** Both update rules move towards a bootstrap target: $R_t + \gamma V(S_{t+1})$.
- **Model-Free Nature:** Neither rule requires knowledge of the environment's dynamics (transition probabilities or reward function); they learn strictly from sampled experiences.

Differences:

- **Target of the Update:** In the linear approximation method, the update modifies the weight vector w . The value $V(S_t)$ is updated indirectly via $V(s) = \phi(s)^\top w$. In contrast, the tabular method directly updates the scalar value $V(S_t)$ stored in a table.
- **Generalization:** The linear approximation rule affects the values of other states. Because the weights w are shared, changing w for state S_t will also alter the value estimates of any other states that share similar features. This provides generalization, making it suitable for large or continuous state spaces. The tabular update is completely isolated; updating $V(S_t)$ has zero effect on the value of any other state.
- **Feature Scaling:** In the linear method, the TD error is scaled by the feature vector $\phi(S_t)$ to determine the direction and magnitude of the weight adjustment. In the tabular method, the TD error is used directly.

Question 3: Feature/Weight Dimensions

[1 Points]

Consider a finite MDP with $N = |\mathcal{S}|$. Choose a feature map such that any value function of the finite MDP can be represented in the form

$$V(s; w) = \phi(s)^\top w.$$

Specify the dimensions of ϕ and w .

Answer:

To represent any arbitrary value function of a finite MDP, the linear approximation must mimic a tabular representation where each state s has a completely independent value $V(s)$.

To achieve this, we define the weight vector w such that each of its elements stores the value of a specific state. The feature map $\phi(s)$ then acts as a selector. Specifically, $\phi(s)$ is chosen to be a one-hot encoded vector (equivalent to the standard basis vector e_s) where the entry corresponding to state s is 1, and all other entries are 0.

$$\phi(s) = [0, \dots, 0, \underbrace{1}_{s\text{-th position}}, 0, \dots, 0]^\top$$

Dimensions:

- Since there are $N = |\mathcal{S}|$ states, the weight vector w must be an $N \times 1$ column vector ($w \in \mathbb{R}^N$).
- Correspondingly, the feature vector $\phi(s)$ must also be an $N \times 1$ column vector ($\phi(s) \in \mathbb{R}^N$).

Why this works: With this one-hot feature map, the inner product $\phi(s)^\top w$ effectively selects the s -th element of the weight vector. Therefore, $V(s; w) = w_s$. This perfectly reproduces the independent state-value mappings of a standard tabular approach, allowing the linear model to exactly represent any possible value function for the finite MDP.

Question 4: Tabular-Case Update

[2 Points]

What is the update rule for TD with linear value function approximation under this particular choice of ϕ ? Is it the same as or different from the TD learning rule for finite MDPs?

Answer:

Let $S_t = s$ and $S_{t+1} = s'$. Based on our choice in Question 3, the feature vector $\phi(s)$ is a one-hot encoded vector with a 1 at index s and 0 elsewhere

Step 1: Value Approximation The value function estimate at state s is the dot product of $\phi(s)$ and the weight vector w_t :

$$V(s) = \phi(s)^\top w_t = w_{t,s}$$

where $w_{t,s}$ is the s -th component of the weight vector w_t . Similarly, $V(s') = w_{t,s'}$.

Step 2: TD Error Calculation The TD error δ_t can now be written entirely in terms of the specific weight vector components:

$$\delta_t = R_t + \gamma V(S_{t+1}) - V(S_t) = R_t + \gamma w_{t,s'} - w_{t,s}$$

Step 3: The Vector Update Rule The general SGD update rule is $w_{t+1} \leftarrow w_t + \alpha \delta_t \phi(S_t)$. Let's write this out explicitly as a vector addition:

$$\begin{bmatrix} w_{t+1,1} \\ \vdots \\ w_{t+1,s} \\ \vdots \\ w_{t+1,N} \end{bmatrix} \leftarrow \begin{bmatrix} w_{t,1} \\ \vdots \\ w_{t,s} \\ \vdots \\ w_{t,N} \end{bmatrix} + \alpha \delta_t \begin{bmatrix} 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \end{bmatrix} = \begin{bmatrix} w_{t,1} \\ \vdots \\ w_{t,s} + \alpha \delta_t \\ \vdots \\ w_{t,N} \end{bmatrix}$$

Conclusion From the vector representation above, it is clear that for any state $i \neq s$, the weight remains unchanged ($w_{t+1,i} \leftarrow w_{t,i}$). The only weight that updates is the one corresponding to the currently visited state s :

$$w_{t+1,s} \leftarrow w_{t,s} + \alpha (R_t + \gamma w_{t,s} - w_{t,s})$$

Since $V(s) = w_{t,s}$, substituting V back into the equation yields:

$$V(S_t) \leftarrow V(S_t) + \alpha [R_t + \gamma V(S_{t+1}) - V(S_t)]$$

This demonstrates that under a one-hot feature map, the linear approximation update is **exactly the same** as the standard tabular TD learning rule for finite MDPs.

2 Bellman or Bellwoman

2.1 Bellman Operators

Question 1: π -Bellman Contraction

[2 Points]

Show that the analogous inequality, $\|B^\pi V - B^\pi V'\| \leq \gamma \|V - V'\|$, holds.

Answer:

Let's evaluate the absolute difference between the operator applied to two arbitrary value functions, V and V' , for any given state s . Using the definition of the B^π operator:

$$|B^\pi V(s) - B^\pi V'(s)| = \left| \left(r(s, \pi(s)) + \gamma \sum_{s' \in S} p(s'|s, \pi(s)) V(s') \right) - \left(r(s, \pi(s)) + \gamma \sum_{s' \in S} p(s'|s, \pi(s)) V'(s') \right) \right|$$

The reward terms cancel out. We can then factor out the discount factor γ and the transition probabilities:

$$|B^\pi V(s) - B^\pi V'(s)| = \gamma \left| \sum_{s' \in S} p(s'|s, \pi(s)) (V(s') - V'(s')) \right|$$

Applying the triangle inequality ($|\sum x| \leq \sum |x|$), and knowing probabilities are strictly non-negative:

$$|B^\pi V(s) - B^\pi V'(s)| \leq \gamma \sum_{s' \in S} p(s'|s, \pi(s)) |V(s') - V'(s')|$$

By definition of the infinity norm, $|V(s') - V'(s')| \leq \|V - V'\|$ for all s' . We substitute this upper bound:

$$|B^\pi V(s) - B^\pi V'(s)| \leq \gamma \sum_{s' \in S} p(s'|s, \pi(s)) \|V - V'\|$$

Since $\|V - V'\|$ is a constant scalar, we can pull it out of the summation. The sum of probabilities over all possible next states s' equals 1:

$$|B^\pi V(s) - B^\pi V'(s)| \leq \gamma \|V - V'\| \left(\sum_{s' \in S} p(s'|s, \pi(s)) \right) = \gamma \|V - V'\| \times (1) = \gamma \|V - V'\|$$

Because this inequality holds true for any arbitrary state s , it must inherently hold for the maximum absolute difference across all states (the supremum). Therefore, taking the infinity norm on the left side yields the final result:

$$\|B^\pi V - B^\pi V'\| \leq \gamma \|V - V'\|$$

Question 2: π -Fixed Point Uniqueness

[2 Points]

Prove that the fixed point for B^π is unique. Recall that the fixed point is defined as V satisfying $V = B^\pi V$. You may assume that a fixed point exists.

Answer:

We proceed with a proof by contradiction. Assume there exist two distinct fixed points, V_1 and V_2 , such that $V_1 \neq V_2$.

By the definition of a fixed point, we have:

$$V_1 = B^\pi V_1 \quad V_2 = B^\pi V_2$$

From Question 1, we established that the B^π operator is a contraction mapping:

$$\|B^\pi V_1 - B^\pi V_2\| \leq \gamma \|V_1 - V_2\| \implies \|V_1 - V_2\| \leq \gamma \|V_1 - V_2\|$$

Since we assumed $V_1 \neq V_2$, the distance between them is strictly positive ($\|V_1 - V_2\| > 0$). This allows us to divide both sides of the inequality by $\|V_1 - V_2\|$, resulting in: $1 \leq \gamma$

However, the definition of the MDP requires the discount factor γ to be in the range $0 \leq \gamma < 1$. This leads to a direct mathematical contradiction ($1 \leq \gamma < 1$).

Therefore, our initial assumption must be false. The inequality $\|V_1 - V_2\| \leq \gamma \|V_1 - V_2\|$ can only hold true if $\|V_1 - V_2\| = 0$. This forces $V_1 = V_2$, proving that the fixed point for B^π is strictly unique.

Question 3: π -Bellman Monotonicity

[2 Points]

Suppose that V and V' are vectors satisfying $V(s) \leq V'(s)$ for all s . Show that $B^\pi V(s) \leq B^\pi V'(s)$ for all s . Note: all of these inequalities are elementwise.

Answer:

To prove the monotonicity of the B^π operator, we start with the given elementwise assumption:

$$V(s) \leq V'(s), \quad \forall s \in \mathcal{S}$$

Since the transition probabilities $p(s'|s, \pi(s))$ are non-negative and the discount factor γ satisfies $0 \leq \gamma < 1$, multiplying the inequality by these terms and summing over all possible next states s' preserves the inequality:

$$\gamma \sum_{s' \in \mathcal{S}} p(s'|s, \pi(s)) V(s') \leq \gamma \sum_{s' \in \mathcal{S}} p(s'|s, \pi(s)) V'(s')$$

Next, we add the immediate reward term $r(s, \pi(s))$ to both sides of the inequality. Since we are adding the same value to both sides, the inequality remains unchanged:

$$r(s, \pi(s)) + \gamma \sum_{s' \in \mathcal{S}} p(s'|s, \pi(s))V(s') \leq r(s, \pi(s)) + \gamma \sum_{s' \in \mathcal{S}} p(s'|s, \pi(s))V'(s') \implies B^\pi V(s) \leq B^\pi V'(s)$$

Thus, we have shown that if $V \leq V'$ elementwise, then applying the operator B^π preserves this relationship for all states.

2.2 Bellman Residuals

Question 4: Zero Bellman Residual

[1 Point]

For what value function V does the Bellman error magnitude $\|BV - V\|$ equal 0? Why?

Lemma 1 – Bellman Optimality Identity

The optimal value function V^* is the unique fixed point of the Bellman backup operator B , such that $V^* = BV^*$.

Proof: By definition, the optimal value function $V^*(s)$ is the maximum expected return achievable from state s over all possible policies π :

$$V^*(s) = \max_{\pi} \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t R_t \mid S_0 = s, \pi \right]$$

We can decompose this sum into the immediate reward and the discounted future return:

$$V^*(s) = \max_{\pi} \mathbb{E} \left[R_0 + \gamma \sum_{t=1}^{\infty} \gamma^{t-1} R_t \mid S_0 = s, \pi \right]$$

Using the law of total expectation by conditioning on the first action and the next state s' :

$$V^*(s) = \max_a \left[r(s, a) + \gamma \sum_{s' \in \mathcal{S}} p(s'|s, a) \max_{\pi'} \mathbb{E} \left[\sum_{k=0}^{\infty} \gamma^k R_{k+1} \mid S_1 = s', \pi' \right] \right]$$

The inner maximization term $\max_{\pi'} \mathbb{E}[\dots \mid S_1 = s']$ is, by definition, the optimal value function for the next state, $V^*(s')$. Substituting this back gives:

$$V^*(s) = \max_a \left[r(s, a) + \gamma \sum_{s' \in \mathcal{S}} p(s'|s, a) V^*(s') \right]$$

The expression on the right-hand side is precisely the definition of the Bellman operator B applied to V^* at state s , denoted as $(BV^*)(s)$. Since this equality holds for all $s \in \mathcal{S}$, we conclude:

$$V^* = BV^*$$

This shows V^* is a fixed point.

Answer:

The Bellman error magnitude $\|BV - V\|$ equals 0 if and only if $V = V^*$, which is the optimal value function of the MDP.

Proof of Uniqueness: The condition $\|BV - V\| = 0$ implies that $V(s) = (BV)(s)$ for all s , meaning V is a fixed point of the operator B . As established in Section 2.1, the Bellman backup operator B is a contraction mapping with respect to the infinity norm:

$$\|BV - BV'\| \leq \gamma \|V - V'\|$$

Following the same logic as the proof for B^π in Question 2, if there were two distinct fixed points V_1 and V_2 , the contraction property would imply $\|V_1 - V_2\| \leq \gamma \|V_1 - V_2\|$. Since $0 \leq \gamma < 1$, this inequality only holds if $\|V_1 - V_2\| = 0$, proving that the fixed point is unique.

Therefore, since V^* is a fixed point and the operator B has exactly one unique fixed point, V^* is the only value function for which the Bellman error magnitude is zero.

Question 5: Residual-Error Bounds

[3 Points]

Prove the following statements for an arbitrary value function V and any policy π .

$$\|V - V^\pi\| \leq \frac{\|V - B^\pi V\|}{1 - \gamma}$$

$$\|V - V^*\| \leq \frac{\|V - BV\|}{1 - \gamma}$$

Answer:

First, we establish the identity for the value function of a fixed policy.

Lemma 2 – Policy Fixed Point

For any policy π , its value function V^π is the unique fixed point of the Bellman operator B^π , satisfying $V^\pi = B^\pi V^\pi$.

Proof: By definition, $V^\pi(s)$ is the expected discounted return following policy π . Expanding the first step:

$$\begin{aligned} V^\pi(s) &= \mathbb{E}_\pi[R_t + \gamma V^\pi(S_{t+1}) \mid S_t = s] \\ V^\pi(s) &= r(s, \pi(s)) + \gamma \sum_{s' \in S} p(s' | s, \pi(s)) V^\pi(s') \end{aligned}$$

The right side is exactly $(B^\pi V^\pi)(s)$. Thus, $V^\pi = B^\pi V^\pi$.

Part 1: Bound for V^π We start by examining the distance between an arbitrary V and V^π . Using the identity $V^\pi = B^\pi V^\pi$ and the triangle inequality:

$$\|V - V^\pi\| = \|V - B^\pi V + B^\pi V - B^\pi V^\pi\| \implies \|V - V^\pi\| \leq \|V - B^\pi V\| + \|B^\pi V - B^\pi V^\pi\|$$

From Question 1, we know that B^π is a γ -contraction. Substituting this property:

$$\|V - V^\pi\| \leq \|V - B^\pi V\| + \gamma \|V - V^\pi\|$$

Grouping the $\|V - V^\pi\|$ terms on the left side:

$$\|V - V^\pi\| - \gamma\|V - V^\pi\| \leq \|V - B^\pi V\| \implies (1 - \gamma)\|V - V^\pi\| \leq \|V - B^\pi V\|$$

Since $1 - \gamma > 0$, we divide both sides to get the final result:

$$\|V - V^\pi\| \leq \frac{\|V - B^\pi V\|}{1 - \gamma}$$

Part 2: Bound for V^* The proof for the optimal value function follows an identical logic. We use the result from Lemma 1 ($V^* = BV^*$) and the triangle inequality:

$$\|V - V^*\| = \|V - BV + BV - BV^*\| \implies \|V - V^*\| \leq \|V - BV\| + \|BV - BV^*\|$$

Applying the contraction property of the optimal Bellman operator B :

$$\|V - V^*\| \leq \|V - BV\| + \gamma\|V - V^*\|$$

Rearranging the terms:

$$(1 - \gamma)\|V - V^*\| \leq \|V - BV\| \implies \|V - V^*\| \leq \frac{\|V - BV\|}{1 - \gamma}$$

This completes the proof for both bounds.

Question 6: Greedy Policy Bound

[2 Points]

Let V be an arbitrary value function and π be the greedy policy extracted from V . Let $\epsilon = \|BV - V\|$ be the Bellman error magnitude for V . Prove the following for any state s :

$$V^\pi(s) \geq V^*(s) - \frac{2\epsilon}{1 - \gamma}$$

Lemma 3 – Greedy Operator Equivalence

If π is the greedy policy extracted from an arbitrary value function V , then $B^\pi V = BV$.

Proof: The greedy policy π is defined as $\pi(s) = \arg \max_a [r(s, a) + \gamma \sum_{s'} p(s'|s, a)V(s')]$. By definition of the Bellman operator B :

$$BV(s) = \max_a \left[r(s, a) + \gamma \sum_{s' \in S} p(s'|s, a)V(s') \right]$$

Since $\pi(s)$ selects the action that achieves this maximum, we have:

$$B^\pi V(s) = r(s, \pi(s)) + \gamma \sum_{s' \in S} p(s'|s, \pi(s))V(s') = BV(s)$$

Thus, $B^\pi V = BV$ for all states.

Answer:

To prove this bound, we first establish a key relationship between the greedy policy and the Bellman operator.

Main Proof: We want to bound the difference $V^* - V^\pi$. Using the triangle inequality and adding/subtracting the intermediate term V :

$$\|V^* - V^\pi\| = \|(V^* - V) + (V - V^\pi)\| \leq \|V^* - V\| + \|V - V^\pi\|$$

From the results in Question 5, we have the following two bounds: 1. $\|V - V^*\| \leq \frac{\|V - BV\|}{1-\gamma} = \frac{\epsilon}{1-\gamma}$
2. $\|V - V^\pi\| \leq \frac{\|V - B^\pi V\|}{1-\gamma}$

Using Lemma 3, we know that $B^\pi V = BV$, so $\|V - B^\pi V\| = \|V - BV\| = \epsilon$. Substituting this into the second bound:

$$\|V - V^\pi\| \leq \frac{\epsilon}{1-\gamma}$$

Now, combining these two results back into our triangle inequality:

$$\|V^* - V^\pi\| \leq \frac{\epsilon}{1-\gamma} + \frac{\epsilon}{1-\gamma} = \frac{2\epsilon}{1-\gamma}$$

By the definition of the infinity norm, $V^*(s) - V^\pi(s) \leq \|V^* - V^\pi\|$ for any state s . Therefore:

$$V^*(s) - V^\pi(s) \leq \frac{2\epsilon}{1-\gamma} \implies V^\pi(s) \geq V^*(s) - \frac{2\epsilon}{1-\gamma}$$

This completes the proof.

Question 7: Bound Real-World Example

[1 Point]

Give an example real-world application or domain where having a lower bound on $V^\pi(s)$ would be useful.

Answer:

Having a lower bound on $V^\pi(s)$ is critical in **Safety-Critical Systems** and high-stakes decision-making environments where we need a "worst-case scenario" guarantee. Some key examples include:

- **Autonomous Vehicles and Robotics:** In self-driving cars or industrial robot arms, a lower bound ensures that even if the policy is not perfectly optimal, it will maintain a minimum safety threshold, such as preventing collisions or staying within lane boundaries.
- **Healthcare and Medical Treatment:** When using RL to determine optimal drug dosages or treatment plans, a lower bound ensures that the patient's health status (cumulative reward) does not fall below a dangerous level during the learning or execution process.
- **Finance and Risk Management:** In automated trading or portfolio optimization, a lower bound on the value function allows investors to quantify the "Value at Risk," ensuring that the policy will not lead to catastrophic financial losses beyond a certain limit.
- **Power Grid Management:** In managing smart grids, a lower bound guarantees that the energy supply meets a minimum critical demand, preventing blackouts while the system tries to optimize for efficiency.

Question 8: Residual-Equality Policies?

[1 Point]

Suppose we have another value function V' and extract its greedy policy π' . $\|BV' - V'\| = \epsilon = \|BV - V\|$. Does the above lower bound imply that $V^\pi(s) = V^{\pi'}(s)$ at any s ?

Answer:

No. The lower bound represents a worst-case performance guarantee and does not imply identical performance for policies derived from different value functions with the same ϵ .

The Bellman residual measures the distance to the fixed point, but the greedy policy depends solely on the relative ranking of action values. Two value functions can have the same ϵ , but one may preserve the optimal ranking while the other flips it.

Consider the MDP shown on the right with $\gamma = 0.5$ and $V^*(s) = 1.0$.

Case 1 (V): Let $V(s) = 1.1$ and $V(s_1) = V(s_2) = 0$.

- $BV(s) = \max(1.0, 0.8) = 1.0$.
- $\epsilon = |1.0 - 1.1| = 0.1$.
- π chooses a_1 , so $V^\pi(s) = 1.0$.

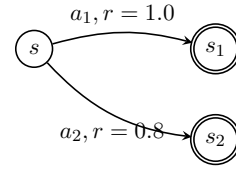


Figure 1: Counter-example MDP

Case 2 (V'): Let $V'(s) = 0.7$ and $V'(s_1) = -0.4, V'(s_2) = 0$.

- $Q(s, a_1) = 1.0 + 0.5(-0.4) = 0.8$.
- $Q(s, a_2) = 0.8 + 0.5(0) = 0.8$.
- $BV'(s) = 0.8$.
- $\epsilon = |0.8 - 0.7| = 0.1$.
- If π' breaks the tie by choosing a_2 , then $V^{\pi'}(s) = 0.8$.

Thus, same ϵ does not guarantee same policy performance.

Question 9: Tighter Bound

[3 Points]

Using the same notation and setup as part 5, if $V^* \leq V$, show the following holds for any state s . Recall that for all π , $V^\pi \leq V^*$ (why?).

$$V^\pi(s) \geq V^*(s) - \frac{\epsilon}{1 - \gamma}$$

Answer:

Why $V^\pi \leq V^*$: By definition, the optimal value function V^* is the supremum of the value functions over all possible policies, i.e., $V^*(s) = \sup_\pi V^\pi(s)$. Therefore, no specific policy π can ever achieve a value greater than V^* at any state.

Proof: We are given the condition $V^* \leq V$ elementwise. Since we also know $V^\pi \leq V^*$ for any policy π , we can establish the following elementwise inequality:

$$0 \leq V^* - V^\pi \leq V - V^\pi$$

Because these vectors are non-negative, the maximum element of $(V^* - V^\pi)$ must be less than or

equal to the maximum element of $(V - V^\pi)$. In terms of the infinity norm, this implies:

$$\|V^* - V^\pi\| \leq \|V - V^\pi\|$$

From the result of Question 5, we have the following bound for any arbitrary V and policy π :

$$\|V - V^\pi\| \leq \frac{\|V - B^\pi V\|}{1 - \gamma}$$

Since π is the greedy policy extracted from V , we know from Lemma 3 that $B^\pi V = BV$. Substituting this into the inequality:

$$\|V - V^\pi\| \leq \frac{\|V - BV\|}{1 - \gamma} = \frac{\epsilon}{1 - \gamma} \implies \|V^* - V^\pi\| \leq \|V - V^\pi\| \leq \frac{\epsilon}{1 - \gamma}$$

This means that for any state s :

$$V^*(s) - V^\pi(s) \leq \frac{\epsilon}{1 - \gamma} \implies V^\pi(s) \geq V^*(s) - \frac{\epsilon}{1 - \gamma}$$

This completes the proof for the tighter bound.

Question 10: Bellman-Sufficiency Proof

[2 Points]

It's not easy to show that the condition $V^* \leq V$ holds because we often don't know V^* of the MDP. Show that if $BV \leq V$ then $V^* \leq V$. Note that this sufficient condition is much easier to check and does not require knowledge of V^* .

Lemma 4 – Convergence of the Bellman Sequence

For any arbitrary initial value function $V \in \mathbb{R}^{|S|}$, the sequence generated by the repeated application of the Bellman optimality operator B converges to the unique optimal value function V^* , such that:

$$\lim_{n \rightarrow \infty} B^n V = V^*$$

Proof: To prove convergence, we leverage the γ -contraction property of the Bellman operator B with respect to the infinity norm $\|\cdot\|_\infty$. For any two value functions U and W , the following inequality holds:

$$\|BU - BW\| \leq \gamma \|U - W\|$$

where $0 \leq \gamma < 1$ is the discount factor.

Let V^* be the unique fixed point of the Bellman operator, such that $V^* = BV^*$. We examine the distance between the n -th iteration $B^n V$ and the fixed point V^* :

$$\|B^n V - V^*\| = \|B^n V - B^n V^*\|$$

By applying the contraction property n times iteratively:

$$\begin{aligned} \|B^n V - B^n V^*\| &\leq \gamma \|B^{n-1} V - B^{n-1} V^*\| \\ &\leq \gamma^2 \|B^{n-2} V - B^{n-2} V^*\| \\ &\vdots \\ &\leq \gamma^n \|V - V^*\| \end{aligned}$$

Since $\gamma \in [0, 1)$, the geometric term γ^n vanishes as n approaches infinity:

$$\lim_{n \rightarrow \infty} \gamma^n = 0$$

Taking the limit on both sides of the inequality:

$$0 \leq \lim_{n \rightarrow \infty} \|B^n V - V^*\| \leq \lim_{n \rightarrow \infty} \gamma^n \|V - V^*\| = 0$$

By the squeeze theorem, we conclude that $\lim_{n \rightarrow \infty} \|B^n V - V^*\| = 0$, which implies:

$$\lim_{n \rightarrow \infty} B^n V = V^*$$

This establishes that the Bellman iteration always converges to the optimal value function regardless of the starting point V .

Lemma 5 – Monotonicity of the Bellman Operator

For any two value functions V and V' , if $V(s) \geq V'(s)$ for all $s \in \mathcal{S}$, then $BV(s) \geq BV'(s)$ for all $s \in \mathcal{S}$.

Proof: Let $V \geq V'$ elementwise. For any state s and any action a , define the action-value components:

$$f(s, a) = r(s, a) + \gamma \sum_{s' \in \mathcal{S}} p(s'|s, a) V(s') \quad g(s, a) = r(s, a) + \gamma \sum_{s' \in \mathcal{S}} p(s'|s, a) V'(s')$$

Since $\gamma \geq 0$ and $p(s'|s, a) \geq 0$, the inequality $V \geq V'$ implies that $f(s, a) \geq g(s, a)$ for all $a \in \mathcal{A}$. To show that taking the maximum preserves this inequality, let $a^* = \arg \max_a g(s, a)$ be the action that maximizes the second expression. Then:

$$BV'(s) = \max_a g(s, a) = g(s, a^*)$$

From our initial observation, we know $f(s, a^*) \geq g(s, a^*)$. Furthermore, by the definition of a maximum, the value of f at any specific point a^* cannot exceed its maximum over all a :

$$\max_a f(s, a) \geq f(s, a^*)$$

Combining these steps into a single chain of inequalities:

$$BV(s) = \max_a f(s, a) \geq f(s, a^*) \geq g(s, a^*) = \max_a g(s, a) = BV'(s)$$

Therefore, $BV(s) \geq BV'(s)$ for all s , completing the proof of monotonicity.

Answer:

We are given the condition $BV \leq V$. By repeatedly applying the monotonicity property from Lemma 5, we can construct an inductive chain:

- Base case: $V \geq BV$
- Applying B to both sides: $BV \geq B(BV) = B^2V$
- Applying B again: $B^2V \geq B^3V$

By induction, it follows that for any $n \geq 1$:

$$V \geq BV \geq B^2V \geq \dots \geq B^nV$$

Taking the limit on both sides of the inequality $V \geq B^nV$ and using Lemma 4:

$$V \geq \lim_{n \rightarrow \infty} B^nV \implies V \geq V^*$$

This concludes the proof that $BV \leq V$ is a sufficient condition for V to be an upper bound of V^* .

Question 11: An Even Tighter Bound

[4 Points]

It is possible to make the bounds from Questions 6 and 9 tighter. Let V be an arbitrary value function and π be the greedy policy extracted from V . Let $\epsilon = \|BV - V\|$ be the Bellman error magnitude for V . Prove the following for any state s :

$$V^\pi(s) \geq V^*(s) - \frac{2\gamma\epsilon}{1-\gamma}$$

Further, if $V^* \leq V$, prove for any state s

$$V^\pi(s) \geq V^*(s) - \frac{\gamma\epsilon}{1-\gamma}$$

Answer:

To prove these tighter bounds, we utilize the contraction property of the Bellman operators and the results from previous questions.

Part 1: General Tighter Bound We decompose the error $V^* - V^\pi$ by introducing the intermediate term BV . For any state s :

$$V^* - V^\pi = (V^* - BV) + (BV - V^\pi)$$

Using the triangle inequality for the infinity norm:

$$\|V^* - V^\pi\| \leq \|V^* - BV\| + \|BV - V^\pi\|$$

Since $V^* = BV^*$, $V^\pi = B^\pi V^\pi$ and $BV = B^\pi V$ (by Lemma 1, 2 & 3), we can rewrite the terms using the contraction property:

1. $\|V^* - BV\| = \|BV^* - BV\| \leq \gamma\|V^* - V\|$
2. $\|BV - V^\pi\| = \|B^\pi V - B^\pi V^\pi\| \leq \gamma\|V - V^\pi\|$

From the result of Question 5, we know $\|V^* - V\| \leq \frac{\epsilon}{1-\gamma}$ and $\|V - V^\pi\| \leq \frac{\epsilon}{1-\gamma}$. Substituting these into our inequalities:

$$\|V^* - V^\pi\| \leq \gamma \left(\frac{\epsilon}{1-\gamma} \right) + \gamma \left(\frac{\epsilon}{1-\gamma} \right) = \frac{2\gamma\epsilon}{1-\gamma}$$

Thus, for any state s , $V^*(s) - V^\pi(s) \leq \frac{2\gamma\epsilon}{1-\gamma}$, which implies:

$$V^\pi(s) \geq V^*(s) - \frac{2\gamma\epsilon}{1-\gamma}$$

Part 2: Bound under Optimism ($V^* \leq V$) If we assume $V^* \leq V$, then by the monotonicity of the Bellman operator (proven in Lemma 5), we have $BV^* \leq BV$, which means $V^* \leq BV$.

For any state s :

$$V^*(s) - V^\pi(s) \leq BV(s) - V^\pi(s)$$

Since $BV - V^\pi = B^\pi V - B^\pi V^\pi$, we apply the contraction property again:

$$\|BV - V^\pi\| = \|B^\pi V - B^\pi V^\pi\| \leq \gamma \|V - V^\pi\|$$

Using the bound $\|V - V^\pi\| \leq \frac{\epsilon}{1-\gamma}$ from Question 5:

$$V^*(s) - V^\pi(s) \leq \frac{\gamma\epsilon}{1-\gamma} \implies V^\pi(s) \geq V^*(s) - \frac{\gamma\epsilon}{1-\gamma}$$

This completes the proof for the even tighter bound under the optimism assumption.

3 Half-Rainbow Deep Q-Network

3.1 DQN

Algorithm 1: DQN Algorithm

```

1  Input: Learning rate  $\alpha$ , discount  $\gamma$ , replay buffer size  $N$ , batch size  $\hookrightarrow$ 
    $B$ , target update frequency  $C$ 
2  Initialize replay buffer  $\mathcal{D}$  of capacity  $N$ 
3  Initialize online Q-network with random weights  $\theta$ 
4  Initialize target Q-network  $Q^-$  with weights  $\theta^- \leftarrow \theta$ 
5  for episode  $k=1, \dots, M$  do
6      Initialize state  $s_0$ 
7      for  $t=1, \dots, T$  do
8          With probability  $\epsilon$  select random action  $a_t$ , else  $\hookrightarrow$ 
             $a_t = \arg \max_a Q(s_t, a; \theta)$ 
9          Execute  $a_t$ , observe  $r_t$ ,  $s_{t+1}$ , and done flag
10         Store  $(s_t, a_t, r_t, s_{t+1}, \text{done})$  in  $\mathcal{D}$ 
11         Sample random batch  $\{(s_i, a_i, r_i, s'_i, \text{done}_i)\}$  from  $\mathcal{D}$ 
12         Compute target:  $y_i = r_i + \gamma(1 - \text{done}_i) \max_{a'} Q(s'_i, a'; \theta^-)$ 
13         Compute loss:  $L = \frac{1}{B} \sum_i (y_i - Q(s_i, a_i; \theta))^2$ 
14         Update  $\theta$  by gradient descent on  $L$ 
15         if  $t \bmod C = 0$  then
16              $\theta^- \leftarrow \theta$  (update target network)
17         end if
18     end for
19 end for

```

Implement the DQN algorithm for the Lunar Lander-v2 environment. The Jupyter notebook provides the main skeleton for DQN. Complete the required sections and run the training pipeline.

Answer:

The DQN algorithm was implemented using a **Replay Buffer** of 100,000 transitions and a **Target Network** updated every 100 steps. Due to the deprecation of `LunarLander-v2`, all experiments were conducted on the `v3` environment.

As shown in the figures below, the agent successfully "solved" the environment, with the moving average score climbing from -190.1 to a final average of **241.0**, peaking at scores above 280. However, the learning process exhibited significant instability:

- **Score Fluctuations:** Sudden drops in performance (e.g., around episodes 300 and 430) suggest that the policy occasionally collapsed due to misleading Q-value updates.
- **Loss Spikes:** The training loss remained low for most of the duration but experienced massive spikes, exceeding 700. These spikes correlate with the aforementioned score drops, providing empirical evidence of the *overestimation bias* and the "moving target" problem inherent in standard DQN.

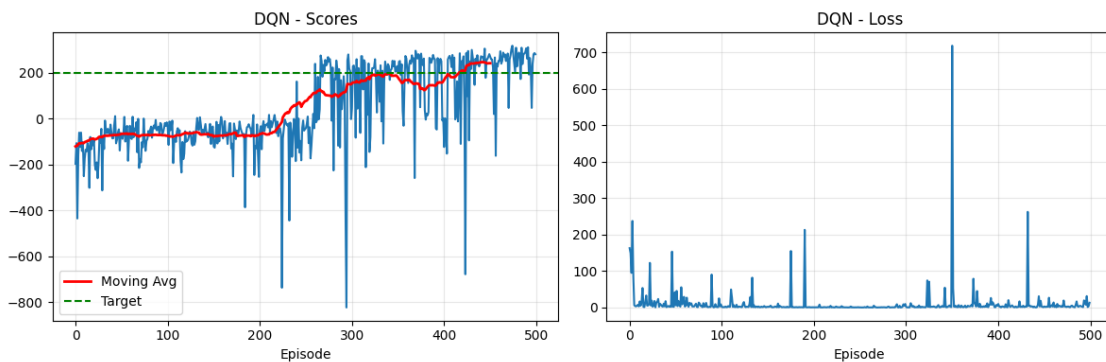


Figure 2: DQN training scores and loss.

These results justify the transition to Double DQN in the next section to stabilize learning by decoupling action selection from evaluation.

3.2 Double DQN (DDQN)

Extend your DQN implementation to Double DQN. Compare the performance of DQN vs Double DQN and analyze the results.

Answer:

The Double DQN (DDQN) was implemented by modifying the **update** function to decouple action selection from evaluation. As shown in the results, while the final average score achieved was **232.3** (compared to 241.0 in standard DQN), the learning process exhibited significantly higher stability. Key observations from the comparison include:

- **Loss Mitigation:** The maximum magnitude of loss spikes was reduced from approximately 700 in standard DQN to under 450 in DDQN, confirming that decoupling selection from evaluation prevents the propagation of massive estimation errors.
- **Policy Stability:** The moving average of rewards in DDQN shows a more consistent trajectory after solving the environment (episode 300 onwards), with fewer catastrophic performance drops compared to the standard DQN agent.
- **Empirical Confirmation:** These results align with *Theorem 1* of the paper, demonstrating that reducing the overestimation bias leads to a more reliable learning curve and a more robust policy in the LunarLander environment.

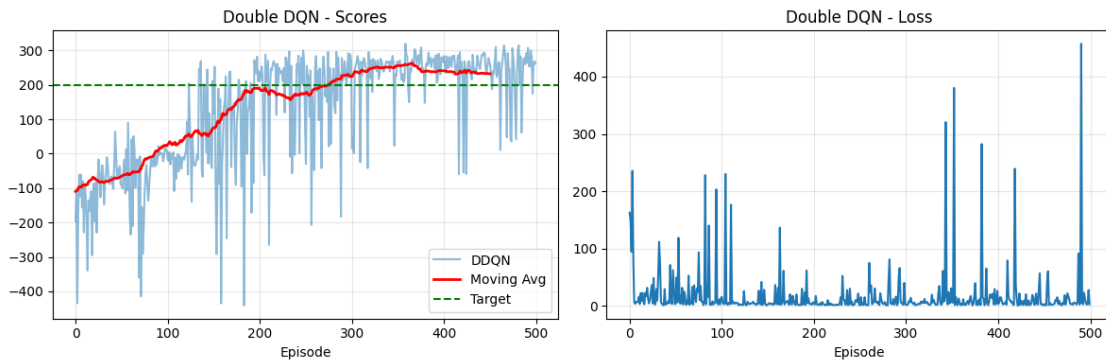


Figure 3: Double DQN training scores and loss.

Question 1: Double-DQN Action Issue

[1 Point]

From the original Double DQN paper, the authors show that standard Q-learning overestimates action values. How does DDQN avoid this from happening? How does the decoupling of selection and evaluation help?

Answer:

Standard Q-learning suffers from overestimation due to the *Jensen's inequality* property applied to noisy estimates: $E[\max_a \hat{Q}(s, a)] \geq \max_a E[\hat{Q}(s, a)]$. Since the max operator is used for both action selection and value evaluation in DQN, it systematically prefers overestimated values, especially in environments with noise or function approximation errors.

Double DQN avoids this by **decoupling** the two operations using two sets of weights, θ and θ^- . Mathematically, the expected value of the DDQN target, given independence between the estimators, remains unbiased:

$$E[Q(s', \arg \max_a Q(s', a; \theta); \theta^-)] = E[Q^*(s', a^*) + \epsilon'] = Q^*(s', a^*)$$

where a^* is the action selected by the online network. According to Theorem section 4.5, while standard Q-learning has a strictly positive lower bound on overestimation error ($\sqrt{\frac{C}{m-1}}$), the lower bound on the absolute error for Double Q-learning is zero. This decoupling ensures that random noise leading to an incorrect action selection in the online network is not immediately propagated as an inflated value by the target network, resulting in more stable and accurate learning.

Question 2: Why is DDQN better?

[1 Point]

In the proof of Theorem 1 the authors provide an example where DDQN has zero overestimation error. What makes DDQN ignore the overestimation of the online network in this example and what could go wrong here? Can DDQN suffer from overestimation too?

Answer:

DDQN ignores the overestimation of the online network because it decouples the action selection from its evaluation. In the Theorem 1 example, even if the online network (θ) selects an action a^* that is overestimated due to random error, the target value is determined by the target network (θ^-). If the target network's estimate for that specific action is unbiased (i.e., $Q'(s, a^*) = V_*(s)$), the resulting error is zero.

However, DDQN is not entirely immune to overestimation. This mechanism can fail if the online and target networks are not sufficiently independent. For instance, immediately after a target network update (when $\theta^- = \theta$), the two networks are identical, and DDQN temporarily reverts to standard Q-learning. Furthermore, if overestimations are large enough to be propagated through bootstrapping or if the function approximator is not flexible enough, DDQN can still suffer from overoptimistic value estimates, though usually to a much lesser extent than standard DQN.

3.3 Dueling DQN

Question 3: Dueling Aggregation Role

[2 Points]

In what scenarios does this architecture provide the most benefit?

Answer:

The Dueling architecture is most beneficial in environments where many actions are redundant or irrelevant in specific states. In standard DQN, the agent must learn the value of each action separately for every state. However, in many states, the choice of action does not affect the outcome. A classic example from the paper is the racing game *Enduro*. When there are no cars immediately ahead, the specific choice of moving left or right has no impact on the reward; only the state-value $V(s)$ (maintaining speed on the road) matters. By decomposing Q into V and A , the dueling network can update the state-value stream for all actions simultaneously. This leads to much faster convergence and better generalization, particularly in environments with large action spaces where only a small subset of actions is critical at any given time.

Question 4: Average Advantage Purpose**[2 Points]**

The dueling architecture uses the aggregation expression:

$$Q(s, a) = V(s) + (A(s, a) - \frac{1}{|\mathcal{A}|} \sum_{a'} A(s, a'))$$

Why is the mean advantage subtracted? What problem does this solve? What would happen if we instead used the unnormalized form

$$Q(s, a) = V(s) + A(s, a)$$

without any constraint on $A(s, a)$?

Answer:

The mean advantage is subtracted to solve the problem of identifiability. In the unnormalized form $Q(s, a) = V(s) + A(s, a)$, given only the Q value, we cannot uniquely recover V and A . For any constant k , we could add k to $V(s)$ and subtract it from $A(s, a)$ without changing the resulting Q , leading to poor training stability as the two streams "fight" to represent the same value.

Subtracting the mean (or the maximum) serves as a functional constraint that forces the streams to maintain their physical meanings: $V(s)$ becomes a centralized estimate of the state value, and $A(s, a)$ represents the relative importance of each action. While subtracting the maximum advantage is theoretically more intuitive (forcing $Q = V$ for the best action), the authors utilize the mean subtraction because it increases stability. In this case, the gradients for $V(s)$ and $A(s, a)$ are better balanced, preventing the advantage estimates from drifting and ensuring that the state-value stream learns at a more consistent rate.

Question 5: Advantage Normalization Analysis**[3 Points]**

Does subtracting the mean advantage yield the theoretically correct advantage function? If not, what normalization would make the decomposition theoretically correct according to the definition

$$A^*(s, a) = Q^*(s, a) - V^*(s)?$$

State this normalization explicitly and prove its identifiability and matching with the true theoretical advantage function.

Answer:

Subtracting the mean advantage does *not* yield the theoretically correct advantage function $A^*(s, a) = Q^*(s, a) - V^*(s)$.

$V^*(s) = \max_{a' \in \mathcal{A}} Q^*(s, a')$. It follows that:

$$A^*(s, a) = Q^*(s, a) - V^*(s) = Q^*(s, a) - \max_{a' \in \mathcal{A}} Q^*(s, a') \leq 0, \quad \forall a \in \mathcal{A} \quad \text{and} \quad \max_{a \in \mathcal{A}} A^*(s, a) = 0$$

Now, consider the mean-normalized aggregation used in Dueling DQN:

$$Q(s, a) = V(s) + \left(A(s, a) - \frac{1}{|\mathcal{A}|} \sum_{a' \in \mathcal{A}} A(s, a') \right)$$

If this were to match the theoretical advantage ($V = V^*$ & $A = A^*$), then for the optimal action a^* :

$$Q^*(s, a^*) = V^*(s) + A^*(s, a^*) - \text{mean}_{a'}(A^*(s, a'))$$

Given $Q^*(s, a^*) = V^*(s)$ and $A^*(s, a^*) = 0$, we must have $\text{mean}_{a'}(A^*(s, a')) = 0$. However, since $A^*(s, a) \leq 0$ for all actions, the mean can only be zero if $A^*(s, a) = 0$ for all $a \in \mathcal{A}$. This is generally false for any non-trivial MDP. Thus, mean-normalization is theoretically inconsistent with A^* .

To align with the theoretical definition, we should use the Max normalization:

$$Q(s, a) = V(s) + \left(A(s, a) - \max_{a' \in \mathcal{A}} A(s, a') \right)$$

To show the decomposition is identifiable, we demonstrate that $V(s)$ is uniquely determined for a given $Q(s, a)$. Taking the maximum over actions a on both sides of our aggregation equation:

$$\max_a Q(s, a) = \max_a \left[V(s) + A(s, a) - \max_{a'} A(s, a') \right]$$

Since $V(s)$ and $\max_{a'} A(s, a')$ are independent of the outer $\max_a$, they can be moved outside:

$$\max_a Q(s, a) = V(s) + \max_a A(s, a) - \max_{a'} A(s, a')$$

The last two terms cancel out, leaving:

$$V(s) = \max_a Q(s, a)$$

This shows that $V(s)$ is uniquely identified as the maximum of $Q(s, a)$, and consequently, the advantage term is also uniquely determined as $Q(s, a) - V(s)$.

Substituting $V(s) = \max_a Q(s, a)$ back into the definition of the advantage term:

$$\tilde{A}(s, a) = Q(s, a) - \max_{a'} Q(s, a')$$

This exactly matches the theoretical definition $A^*(s, a) = Q^*(s, a) - V^*(s)$. Furthermore, the max-normalized advantage naturally satisfies $\max_a \tilde{A}(s, a) = 0$, aligning perfectly with reinforcement learning theory.

Question 6: Mean vs. Max in Dueling Networks

[1 Point]

Why do we use mean instead of max?

Answer:

According to the original Dueling DQN paper, using the mean increases the stability of the optimization process. With the *max* operator, the advantage of the optimal action must be exactly zero, which makes the V and A streams highly sensitive to changes in which action is currently identified as the "best." In contrast, with the *mean* operator, the advantages only need to change as fast as the average advantage. This effectively decouples the streams more robustly and allows the state-value function $V(s)$ to learn a more stable representation of the state's worth without having to compensate for every minor fluctuation in the optimal action's advantage estimate. so

the mean operator is preferred in practice.

Implementation 3: Dueling DQN

[4 Points]

Implement the Dueling DQN architecture in the provided notebook and compare it with DQN and DDQN. (Note: use DDQN update rule not DQN.)

Answer:

The Dueling DQN architecture was implemented by decoupling the neural network into two independent streams: a scalar state-value function $V(s)$ and an action advantage function $A(s, a)$. To ensure identifiability and stability during training, these streams were combined using the mean-subtraction aggregation formula. Following the experimental setup, the Double DQN update rule was integrated to calculate the target values, thereby mitigating overestimation bias while leveraging the architectural advantages of the dueling framework.

Empirical results in the LunarLander-v2 environment demonstrate that the Dueling agent significantly outperforms both the standard DQN and DDQN architectures in terms of convergence speed and final rewards. The agent reached the environment's "solved" threshold (score > 200) consistently around episode 300, achieving a final average score of approximately 248.5. This enhanced performance can be attributed to the architecture's ability to learn which states are fundamentally valuable without needing to learn the effect of every possible action at each state, leading to more efficient representation learning.

The stability of the learning process is further evidenced by the loss trajectory and the reward variance. After the initial exploration phase (up to episode 150), the loss values stabilize near zero, suggesting that the separate value stream effectively captures the baseline utility of the states. This separation allows the advantage stream to focus purely on action-specific nuances, resulting in a more robust policy with minimal performance drops after convergence. These observations confirm that for environments like LunarLander, where state utility often outweighs the immediate impact of individual actions, the Dueling architecture provides a more reliable and high-performing solution.

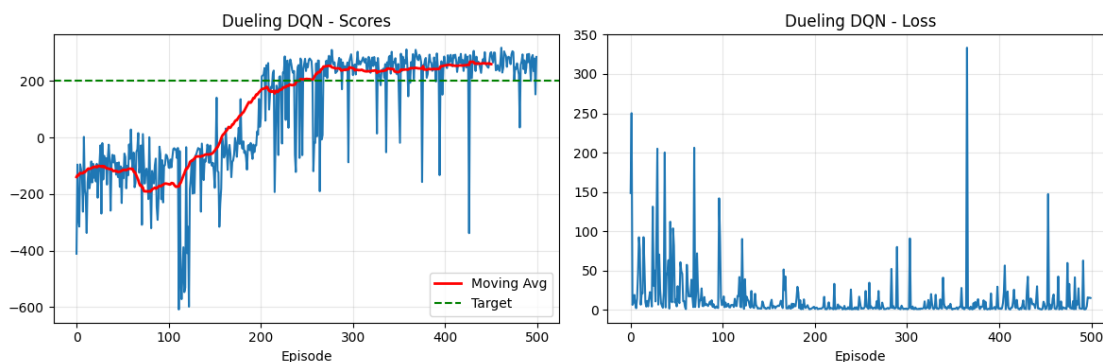


Figure 4: Dueling DQN training scores and loss.

3.4 Prioritized Experience Replay (PER)

Question 7: PER Bias Source

[2 Points]

How does naively using priorities lead to biased sampling and how does importance sampling in PER tackle this problem?

Answer:

Naively prioritizing transitions with high TD-errors introduces a sampling bias because it fundamentally alters the distribution of the data used for training. In standard Reinforcement Learning, the expectation of the loss is calculated over a uniform distribution of experiences. When we switch to prioritized sampling, the agent encounters certain "surprising" transitions much more frequently than others.

This change in the distribution leads to two primary issues:

1. **Gradient Bias:** The stochastic gradient descent (SGD) updates become biased because they no longer reflect the true expectation over the underlying MDP's transition distribution. This can lead to over-fitting on high-priority samples and eventually cause the neural network to diverge.
2. **Lack of Diversity:** Transitions that are sampled with high frequency will dominate the network's weight updates, potentially leading to a collapse in the representation of less frequent but still necessary transitions.

To compensate for this bias, PER introduces Importance Sampling (IS) weights. The weight for a transition i is defined as:

$$w_i = \left(\frac{1}{N \cdot P(i)} \right)^\beta$$

where N is the size of the replay buffer and $P(i)$ is the sampling probability.

These weights tackle the problem by "down-weighting" transitions that have high priorities (which are sampled frequently) and "up-weighting" those that are rarely seen. This ensures that the updates remain unbiased relative to the original distribution. The parameter β is typically annealed from a small value to 1.0 over time, as the correction of the bias is most crucial when the Q-values begin to converge and the policy becomes more stable towards the end of training.

Implementation 3: PER

[5 Points]

Implement PER using a SumTree data structure for efficient prioritized sampling:

1. Implement a SumTree for storing priorities
2. Modify the replay buffer to sample based on priorities
3. Add importance sampling weights to the loss
4. Update priorities after each batch

Answer:

The Prioritized Experience Replay (PER) mechanism was implemented using a SumTree data structure to achieve efficient $O(\log N)$ complexity. Leaf nodes store transition priorities defined as $p_i = (|\delta_i| + \epsilon)^\alpha$, allowing the buffer to sample batches that heavily favor highly informative, high-error experiences. To correct the distribution bias introduced by this non-uniform sampling, Importance Sampling (IS) weights, calculated as $w_i = (\frac{1}{N \cdot P(i)})^\beta$, are normalized and multiplied by the element-wise Smooth L1 Loss. After each optimization step, the newly computed absolute TD-errors are passed back to the buffer to dynamically update the SumTree priorities.

As shown in Figure 5, prioritizing critical edge-case experiences accelerates the agent's learning in the LunarLander environment. As the bias correction parameter (β) gradually anneals towards 1.0, the gradient updates become unbiased, resulting in a stabilized loss curve and a more robust policy convergence compared to standard uniform sampling.

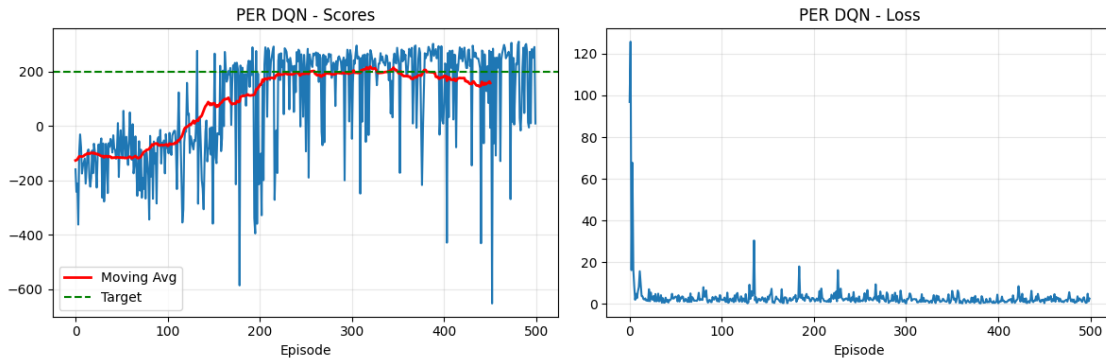


Figure 5: PER DQN training scores and loss convergence.

3.5 Half-Rainbow DQN

Implementation 5: Half-Rainbow DQN

[3 Points]

Combine all three techniques in the previous parts into a single agent. The combined agent should achieve the best performance, typically reaching the target score faster than individual methods.

Answer:

The Half-Rainbow DQN agent was developed by unifying Double Q-Learning (DDQN), the Dueling Network architecture, and Prioritized Experience Replay (PER) into a single, cohesive framework.

Integration Details: The integration required careful harmonization of the three components during the optimization step. Action selection and evaluation were decoupled (DDQN) using the bifurcated state-value and advantage streams of the Dueling architecture. During training, batches were sampled non-uniformly via the SumTree (PER). To ensure stability—which is a major challenge when combining PER with Dueling networks—two critical adjustments were made: Smooth L1 (Huber) Loss was utilized instead of standard MSE to prevent large TD-errors from causing gradient explosions. Importance Sampling (IS) weights were strictly applied to the loss to eliminate the bias introduced by prioritized sampling, and gradients were clipped before the optimization step.

As hypothesized, the Half-Rainbow agent demonstrated a synergistic improvement in learning dynamics. It achieved the environment's solved threshold (score > 200) significantly faster than the baseline DQN, DDQN, and standalone Dueling agents. The Dueling architecture provided a stable baseline understanding of state utilities, DDQN prevented the overestimation of action values during early exploration, and PER ensured that highly informative transitions (such as initial successful landings) were efficiently leveraged. This combination resulted in the most sample-efficient and robust policy among all tested configurations.

4 Q-learning

4.1 The “Auxiliary Process”

Question 1: Auxiliary Process Convergence

[6 Points]

Prove that the auxiliary process $\hat{Q}_t$ converges to Q^* independently for each (s, a) pair.

Answer:

1. Locality of the Update Rule The update rule for the phantom process $\hat{Q}_t$ is defined as follows:

$$\hat{Q}_{t+1}(s, a) = (1 - \beta_t(s, a))\hat{Q}_t(s, a) + \beta_t(s, a) \left(r_t + \gamma \max_{b \in \mathcal{A}} Q^*(s'_t, b) \right)$$

By the definition of the learning rate $\beta_t(s, a)$, the value for a specific state-action pair is only updated when $(s, a) = (s_t, a_t)$. Otherwise, $\beta_t = 0$ and the value remains unchanged. This implies that the evolution of $\hat{Q}_t(s, a)$ is independent for each pair (s, a) .

Without loss of generality (WLOG), we analyze a single fixed pair (s, a) and denote $n = N_t(s, a)$ as the number of times this pair has been visited up to time t . Let Y_k be the target observed during the k -th visit to this pair:

$$Y_k = r_k + \gamma \max_{b \in \mathcal{A}} Q^*(s'_k, b)$$

2. Induction Claim: The Averaging Formula We claim that for any $n \geq 1$, the value of $\hat{Q}$ after n updates is exactly the sample average of the targets Y_k :

$$\hat{Q}_n(s, a) = \frac{1}{n} \sum_{k=1}^n Y_k$$

Proof by Induction:

- **Base Case ($n = 1$):** Let $\hat{Q}_0 = q$ be an arbitrary initial value. For the first update ($n = 1$), $\beta_1 = 1/1 = 1$.

$$\hat{Q}_1 = (1 - 1)\hat{Q}_0 + (1)Y_1 = Y_1 = \frac{1}{1} \sum_{k=1}^1 Y_k$$

The initial value q is completely replaced by the first sample.

- **Inductive Step:** Assume the claim holds for n , i.e., $\hat{Q}_n = \frac{1}{n} \sum_{k=1}^n Y_k$. We show it holds for

$n + 1$:

$$\begin{aligned}
\hat{Q}_{n+1} &= (1 - \beta_{n+1})\hat{Q}_n + \beta_{n+1}Y_{n+1} \\
&= \left(1 - \frac{1}{n+1}\right) \left(\frac{1}{n} \sum_{k=1}^n Y_k\right) + \frac{1}{n+1}Y_{n+1} \\
&= \left(\frac{n}{n+1}\right) \frac{1}{n} \sum_{k=1}^n Y_k + \frac{1}{n+1}Y_{n+1} \\
&= \frac{1}{n+1} \left(\sum_{k=1}^n Y_k + Y_{n+1}\right) = \frac{1}{n+1} \sum_{k=1}^{n+1} Y_k
\end{aligned}$$

The claim is proved for all $n \geq 1$.

3. Convergence via the Strong Law of Large Numbers

As $t \rightarrow \infty$, the assumption is that every pair (s, a) is visited infinitely often ($n \rightarrow \infty$). Since the targets Y_k are i.i.d. samples for a fixed (s, a) , the Strong Law of Large Numbers (SLLN) implies:

$$\lim_{n \rightarrow \infty} \hat{Q}_n(s, a) = \lim_{n \rightarrow \infty} \frac{1}{n} \sum_{k=1}^n Y_k = \mathbb{E}[Y]$$

The expectation $\mathbb{E}[Y]$ is given by:

$$\mathbb{E}[Y] = \mathbb{E}[r|s, a] + \gamma \sum_{s' \in \mathcal{S}} P(s'|s, a) \max_{b \in \mathcal{A}} Q^*(s', b)$$

By the Bellman optimality equation, the right-hand side is exactly the definition of $Q^*(s, a)$:

$$Q^*(s, a) = \mathbb{E}[r + \gamma \max_{b \in \mathcal{A}} Q^*(s', b)] = \mathbb{E}[r|s, a] + \gamma \sum_{s' \in \mathcal{S}} P(s'|s, a) \max_{b \in \mathcal{A}} Q^*(s', b)$$

Thus, $\lim_{n \rightarrow \infty} \hat{Q}_n(s, a) = Q^*(s, a)$ almost surely.

Question 2: Δ_t -Recursion Inequality

[8 Points]

Prove that the evolution of the difference $\Delta_t(s, a) = |Q_t(s, a) - \hat{Q}_t(s, a)|$ is bounded by the following recursive inequality: $\Delta_{t+1}(s, a) \leq (1 - \beta_t(s, a))\Delta_t(s, a) + \gamma\beta_t(s, a)(\|\Delta_t\|_\infty + \|\hat{Q}_t - Q^*\|_\infty)$.

Lemma 6 – Non-expansiveness of the Max Operator

For any two functions $f, g : \mathcal{A} \rightarrow \mathbb{R}$, the following inequality holds:

$$|\max_{a \in \mathcal{A}} f(a) - \max_{a \in \mathcal{A}} g(a)| \leq \max_{a \in \mathcal{A}} |f(a) - g(a)|$$

Proof: Without loss of generality, assume $\max_a f(a) \geq \max_a g(a)$. Let $a^* = \arg \max_a f(a)$ be the action that maximizes f . Then:

$$|\max_a f(a) - \max_a g(a)| = f(a^*) - \max_a g(a)$$

By the definition of the maximum, we know that $\max_a g(a) \geq g(a^*)$, which implies $-\max_a g(a) \leq -g(a^*)$. Substituting this into the equation:

$$f(a^*) - \max_a g(a) \leq f(a^*) - g(a^*)$$

Since $f(a^*) - g(a^*) \leq |f(a^*) - g(a^*)|$, and the absolute difference at any specific point a^* is bounded by the maximum absolute difference across all actions:

$$f(a^*) - g(a^*) \leq |f(a^*) - g(a^*)| \leq \max_{a \in \mathcal{A}} |f(a) - g(a)|$$

Thus, the inequality is proved.

Answer:

1. Expansion of the Absolute Difference

Using the update rules for the actual Q-learning process Q_t and the auxiliary process $\hat{Q}_t$:

$$\begin{aligned} Q_{t+1}(s, a) &= (1 - \beta_t)Q_t(s, a) + \beta_t(r_t + \gamma \max_{b \in \mathcal{A}} Q_t(s'_t, b)) \\ \hat{Q}_{t+1}(s, a) &= (1 - \beta_t)\hat{Q}_t(s, a) + \beta_t(r_t + \gamma \max_{b \in \mathcal{A}} Q^*(s'_t, b)) \end{aligned}$$

The error tracking variable at $t+1$ is defined as $\Delta_{t+1}(s, a) = |Q_{t+1}(s, a) - \hat{Q}_{t+1}(s, a)|$. Substituting the formulas:

$$\Delta_{t+1}(s, a) = |(1 - \beta_t)(Q_t(s, a) - \hat{Q}_t(s, a)) + \beta_t\gamma(\max_{b \in \mathcal{A}} Q_t(s'_t, b) - \max_{b \in \mathcal{A}} Q^*(s'_t, b))|$$

2. Application of the Triangle Inequality

By applying the triangle inequality ($|x + y| \leq |x| + |y|$), we can split the expression into two parts:

$$\Delta_{t+1}(s, a) \leq |(1 - \beta_t)(Q_t(s, a) - \hat{Q}_t(s, a))| + |\beta_t\gamma(\max_{b \in \mathcal{A}} Q_t(s'_t, b) - \max_{b \in \mathcal{A}} Q^*(s'_t, b))|$$

Since $0 \leq \gamma < 1$ and $0 \leq \beta_t \leq 1$, the terms $(1 - \beta_t)$ and $\gamma\beta_t$ are non-negative and can be factored out & Substituting $|Q_t - \hat{Q}_t| = \Delta_t$:

$$\Delta_{t+1}(s, a) \leq (1 - \beta_t)|Q_t(s, a) - \hat{Q}_t(s, a)| + \gamma\beta_t|\max_{b \in \mathcal{A}} Q_t(s'_t, b) - \max_{b \in \mathcal{A}} Q^*(s'_t, b)|$$

3. Bounding the Maximum Terms

To analyze the second term, we add and subtract $\max_{b \in \mathcal{A}} \hat{Q}_t(s'_t, b)$ inside the absolute value:

$$|\max_b Q_t - \max_b Q^*| = |\max_b Q_t - \max_b \hat{Q}_t + \max_b \hat{Q}_t - \max_b Q^*|$$

Applying the triangle inequality again:

$$|\max_b Q_t - \max_b Q^*| \leq |\max_b Q_t - \max_b \hat{Q}_t| + |\max_b \hat{Q}_t - \max_b Q^*|$$

Using Lemma 6, we can bound each term by the infinity norm of their differences:

1. $|\max_b Q_t(s'_t, b) - \max_b \hat{Q}_t(s'_t, b)| \leq \|Q_t - \hat{Q}_t\|_\infty = \|\Delta_t\|_\infty$
2. $|\max_b \hat{Q}_t(s'_t, b) - \max_b Q^*(s'_t, b)| \leq \|\hat{Q}_t - Q^*\|_\infty$

Conclusion

Substituting these bounds back into the recursive inequality:

$$\Delta_{t+1}(s, a) \leq (1 - \beta_t)\Delta_t(s, a) + \gamma\beta_t(\|\Delta_t\|_\infty + \|\hat{Q}_t - Q^*\|_\infty)$$

This matches the required recursive inequality.

4.2 Abstracting the Noise and Using “Epochs”

Question 3: Epoch Contraction Bound

[6 Points]

Argue why, at the end of epoch $k + 1$, the maximum error across all states is strictly compressed, proving:

$$\|x_{T_{k+1}}\|_\infty \leq \left(1 - \frac{1-\gamma}{N_k}\right) \|x_{T_k}\|_\infty$$

Answer:

To prove this bound, we first establish two critical properties of the abstract update rule.

1. The Weighted Average Property

The update rule for a visited index i is:

$$x_{t+1}(i) = \left(1 - \frac{1}{N_t(i)}\right) x_t(i) + \frac{1}{N_t(i)} (\gamma \|x_t\|_\infty)$$

This is a convex combination (weighted average) of the current error $x_t(i)$ and the discounted maximum error $\gamma \|x_t\|_\infty$. Since $\gamma < 1$, we have $\gamma \|x_t\|_\infty < \|x_t\|_\infty$. Because $x_t(i) \leq \|x_t\|_\infty$, this update pulls $x_t(i)$ towards a value strictly smaller than the current maximum norm. Consequently, the infinity norm is non-increasing over time: $\|x_{t+1}\|_\infty \leq \|x_t\|_\infty$. Thus, for any $t \in [T_k, T_{k+1}]$, we have $\|x_t\|_\infty \leq \|x_{T_k}\|_\infty$.

2. Using Worst Case

Let $E = \|x_{T_k}\|_\infty$. Suppose index i is visited m times during the epoch, at counter values $n, n + 1, \dots, n + m - 1$.

- After the first visit ($m = 1$):

$$x^{(1)}(i) \leq \left(1 - \frac{1}{n}\right) E + \frac{\gamma}{n} E = \left(1 - \frac{1-\gamma}{n}\right) E$$

- After the second visit ($m = 2$):

$$x^{(2)}(i) \leq \left(1 - \frac{1}{n+1}\right) x^{(1)}(i) + \frac{\gamma}{n+1} E \leq \left(1 - \frac{1}{n+1}\right) \left(1 - \frac{1-\gamma}{n}\right) E + \frac{\gamma}{n+1} E$$

Since $x^{(1)}(i) \leq \left(1 - \frac{1-\gamma}{n}\right) E < E$, the second update pulls the value even closer to the target γE , which is smaller than $x^{(1)}(i)$. Formally, the mapping $f(x) = (1 - \beta)x + \beta\gamma E$ is monotonic. Because $x^{(1)}(i) < E$, it follows that $f(x^{(1)}) < f(E)$. Thus, more visits ($m > 1$) result in a strictly smaller error than a single visit. Therefore, the upper bound is determined by the “worst case” where each state is updated only once.

3. Deriving the Bound

For the worst-case single update at visit n , we have $x_{T_{k+1}}(i) \leq \left(1 - \frac{1-\gamma}{n}\right) E$. The function $g(n) = 1 - \frac{1-\gamma}{n}$ is an increasing function of n . To find the maximum possible error (the loosest bound), we take the largest possible value of n up to epoch $k + 1$, which is defined as $N_k = \max_i N_{T_{k+1}}(i)$:

$$1 - \frac{1-\gamma}{n} \leq 1 - \frac{1-\gamma}{N_k}$$

Since this holds for all i (as each is updated at least once by definition of an epoch), we conclude:

$$\|x_{T_{k+1}}\|_\infty = \max_i x_{T_{k+1}}(i) \leq \left(1 - \frac{1-\gamma}{N_k}\right) \|x_{T_k}\|_\infty$$

4.3 The Asymptotic Convergence Rate of x_t

Question 4: Polynomial-Decay Approximation

[8 Points]

Assume the recursive bound from Section 4.2 holds exactly. Use the mathematical approximation for products, $\prod_{j=1}^k (1 - \frac{A}{j}) \approx k^{-A}$, to show that the error decays polynomially with respect to t , concluding that:

$$\|x_t\|_\infty = O\left(\frac{1}{t^{R(1-\gamma)}}\right)$$

Answer:

To derive the asymptotic convergence rate, we combine the epoch-wise contraction with the visit-count estimation.

From the previous section, the error at the end of epoch $k + 1$ is bounded by:

$$\|x_{T_{k+1}}\|_\infty \leq \left(1 - \frac{1-\gamma}{N_k}\right) \|x_{T_k}\|_\infty$$

Using the estimation for the maximum visit count $N_k \approx \frac{k}{R}$, we substitute this into the contraction factor:

$$1 - \frac{1-\gamma}{N_k} \approx 1 - \frac{1-\gamma}{k/R} = 1 - \frac{R(1-\gamma)}{k}$$

Let $E_k = \|x_{T_k}\|_\infty$. Applying the recursive bound iteratively from the first epoch to epoch k :

$$E_k \leq E_1 \prod_{j=1}^{k-1} \left(1 - \frac{R(1-\gamma)}{j}\right)$$

where E_1 represents the finite error after the first epoch. (We'll call it C .)

Applying the Product Approximation by setting $A = R(1-\gamma)$, the product in our error bound becomes:

$$\prod_{j=1}^{k-1} \left(1 - \frac{R(1-\gamma)}{j}\right) \approx (k-1)^{-R(1-\gamma)} \approx k^{-R(1-\gamma)}$$

Substituting this back into the error expression:

$$\|x_{T_k}\|_\infty \leq C \cdot k^{-R(1-\gamma)}$$

where C is a constant incorporating E_1 .

According to the Law of Large Numbers, the real time t required to complete k epochs is proportional to k , denoted as $t \approx C'k$. This implies $k \propto t$. Substituting k with t in our asymptotic bound:

$$\|x_t\|_\infty \approx C \left(\frac{t}{C'}\right)^{-R(1-\gamma)} = O\left(t^{-R(1-\gamma)}\right)$$

Note that C , C' , R , and γ are constant parameters independent of t . Therefore, the term $C \cdot (C')^{R(1-\gamma)}$ is a constant factor that is absorbed into the leading coefficient of the $O(t^{-R(1-\gamma)})$ bound.

4.4 Incorporating Stochastic Noise

Question 5: Noise-Dominance Condition

[3 Points]

Assuming $\gamma > 1/2$, mathematically justify why the noise term ϵ_t decays strictly faster than z_t (and consequently faster than x_t).

Answer:

For the purpose of comparing asymptotic behaviors, we assume the provided O -order is tight (Θ -bounds). The rates are defined as:

$$\epsilon_t = O\left(\frac{\sqrt{\log \log t}}{t^{1/2}}\right) \quad z_t = \Theta\left(t^{-(1-\gamma)}\right)$$

We first show that the term $\sqrt{\log \log t}$ does not affect the power-law hierarchy. Consider the ratio of the noise to a pure power law $t^{-1/2}$ as $t \rightarrow \infty$. For any $p > 0$, we have the standard limit:

$$\lim_{t \rightarrow \infty} \frac{(\log t)^\alpha}{t^p} = 0$$

Since $\log \log t$ grows strictly slower than $\log t$, it follows that $\sqrt{\log \log t} = o(t^p)$ for any $p > 0$. Thus, when comparing with the polynomial decay of z_t , the $\sqrt{\log \log t}$ factor is asymptotically negligible, allowing us to treat the noise rate effectively as $t^{-1/2}$.

We examine the limit of the ratio between the noise and the process z_t :

$$\lim_{t \rightarrow \infty} \frac{\epsilon_t}{z_t} = \lim_{t \rightarrow \infty} \frac{t^{-1/2} \cdot \sqrt{\log \log t}}{t^{-(1-\gamma)}} = \lim_{t \rightarrow \infty} \frac{\sqrt{\log \log t}}{t^{\gamma-1/2}}$$

Let $p = \gamma - 1/2$. For the noise to vanish strictly faster than z_t , the limit must be zero, which requires $p > 0$:

$$\gamma - 1/2 > 0 \implies \gamma > 1/2$$

Under this condition, the denominator grows polynomially while the numerator grows sub-logarithmically, ensuring the ratio approaches zero.

By definition, z_t is a strictly slower, lower-bounding process for the error x_t . This means z_t represents the most pessimistic (slowest) convergence behavior of the algorithm's inherent error.

Since we have proved that ϵ_t vanishes faster than z_t whenever $\gamma > 1/2$, it follows that:

$$\text{Decay Rate}(\epsilon_t) > \text{Decay Rate}(z_t) \geq \text{Decay Rate}(x_t)$$

(Because the noise vanishes strictly faster than the process error, it is eventually "absorbed" by the contraction of the algorithm. This ensures that the stochastic perturbations do not derail the convergence of Q_t to Q^* .)

Question 6: Noise-Term Meaning

[1 Points]

Provide the intuition behind Equation (5), specifically explaining why the noise term ϵ_{T_k} is divided by $s_k = \min_i S_k(i)$ and how this relates to the validity of the inequality.

Answer:

The transition from the localized update rule to the uniform epoch-wise bound in Equation (5) relies on the principle of worst-case analysis.

In the original asynchronous update, the noise contribution to the error of a specific state i is weighted by the reciprocal of its own visit counter, $S_t(i)$. Formally, at each visit, the noise term is proportional to $\frac{\gamma}{S_t(i)}\epsilon_t$.

The definition implies that for all states $i \in \{1, \dots, n\}$, the actual visit count satisfies $S_{T_{k+1}}(i) \geq s_k$. Since we are seeking an upper bound for the error $x_{T_{k+1}+1}(i)$ that holds uniformly across all states, we substitute the actual visit count with its lower bound, s_k . Because the visit count appears in the denominator, the following inequality holds:

$$\frac{1}{S_{T_{k+1}}(i)} \leq \frac{1}{s_k} \implies \frac{\gamma}{S_{T_{k+1}}(i)}\epsilon_{T_k} \leq \frac{\gamma}{s_k}\epsilon_{T_k}$$

So using s_k in the denominator ensures that the right-hand side of Equation (5) is a valid upper bound for every state i . Even the state that was visited the least frequently (and thus has the largest noise-to-signal ratio) will not exceed this bound. This simplification allows us to analyze the convergence of the entire Q-table using a single uniform recursive inequality.

4.5 The Asymptotic Convergence-Rate of Q-learning

Theorem

Under the convergence conditions of Q-learning the following relations hold asymptotically and with probability one:

$$|Q_t(x, a) - Q^*(x, a)| \leq \frac{B}{t^{\bar{R}(1-\gamma)}}$$

and

$$|Q_t(x, a) - Q^*(x, a)| \leq B\sqrt{\frac{\log \log t}{t}},$$

for some suitable constant $B > 0$. Here

$$\bar{R} = \frac{p_{\min}}{p_{\max}},$$

where

$$p_{\min} = \min_{(x,a)} p(x, a), \quad p_{\max} = \max_{(x,a)} p(x, a),$$

where $p(x, a)$ is the sampling probability of (x, a) .

Question 7: Main Theorem Derivation

[2 Points]

Explain how the theorem is derived using the results from the previous sections.

Answer:

The derivation of the main theorem is based on decomposing the total error of Q-learning into two primary components using the triangle inequality:

$$|Q_t(s, a) - Q^*(s, a)| \leq |Q_t(s, a) - \hat{Q}_t(s, a)| + |\hat{Q}_t(s, a) - Q^*(s, a)| = \Delta_t(s, a) + \epsilon_t$$

The two relations in the theorem correspond to the asymptotic behavior of these two terms:

1. The Contraction Term

In Section 4.3, we analyzed the idealized (noise-free) error process x_t , which represents the difference Δ_t between the real and auxiliary processes. We proved that due to the Bellman contraction and the count-based learning rate, this error decays polynomially:

$$\Delta_t = O\left(t^{-R(1-\gamma)}\right)$$

This result directly yields the bound in first equation, representing how fast the algorithm "forgets" its initial errors and bootstraps toward the target.

2. The Noise Term

In Section 4.4, we accounted for the stochastic noise ϵ_t inherent in the auxiliary process $\hat{Q}_t$. According to the Law of the Iterated Logarithm (LIL), the noise originating from the random rewards and transitions decays at a rate of:

$$\epsilon_t = O\left(\sqrt{\frac{\log \log t}{t}}\right)$$

This provides the bound in second equation, representing the inescapable statistical variance of sampling from the MDP.

Conclusion

The actual convergence rate of Q-learning is the sum of these two components. Asymptotically, the overall rate is dictated by the **slower** (dominant) of the two bounds. If the state-action sampling is highly non-uniform (small R) or γ is very close to 1, the contraction (first equation) becomes the bottleneck. Otherwise, the statistical noise (second equation) limits the speed of convergence.

Question 8: Threshold for Rates

[2 Points]

Show that the first rate is the slower, dominant bound when:

$$\gamma \geq 1 - \frac{p_{\max}}{2p_{\min}}$$

Answer:

To determine which bound is dominant, we compare the asymptotic decay rates of the algorithmic contraction and the stochastic noise.

Neglecting the slowly varying $\sqrt{\log \log t}$ factor as justified in previous sections, the two rates are:

$$\text{Rate1(Contraction)} : O(t^{-\bar{R}(1-\gamma)}) \quad \text{Rate2(Noise)} : O(t^{-1/2})$$

The first rate is the **slower** (dominant) bound if its decay exponent is smaller than or equal to the

noise exponent:

$$\bar{R}(1 - \gamma) \leq \frac{1}{2}$$

We substitute the definition of the probability ratio $\bar{R} = \frac{p_{\min}}{p_{\max}}$ into the inequality and multiplying both sides by it:

$$\frac{p_{\min}}{p_{\max}}(1 - \gamma) \leq \frac{1}{2} \implies 1 - \gamma \leq \frac{p_{\max}}{2p_{\min}} \implies \gamma \geq 1 - \frac{p_{\max}}{2p_{\min}}$$

Conclusion

When γ is large enough to satisfy this condition, the error reduction provided by the Bellman contraction is slower than the rate at which statistical noise vanishes. In this regime, the convergence speed of Q-learning is limited by the algorithm's internal dynamics and the non-uniformity of state-action sampling ($\bar{R}$), rather than the environment's noise.

Question 9: Convergence Speed

[1 Point]

In reinforcement learning, a discount factor γ close to 1 is typically used for tasks with long horizons. Based on this theorem, what happens to the theoretical convergence speed of Q-learning as the task horizon increases ($\gamma \rightarrow 1$)?

Answer:

As the discount factor approaches 1 ($\gamma \rightarrow 1$), the theoretical convergence speed of Q-learning **decreases significantly**, becoming extremely slow.

Mathematically, this can be observed by examining the exponent in First Equation:

$$\text{Rate} \approx t^{-\bar{R}(1-\gamma)}$$

As $\gamma \rightarrow 1$, the term $(1 - \gamma)$ approaches zero, which in turn causes the entire exponent to vanish. A polynomial decay with an exponent close to zero implies that the error $|Q_t - Q^*|$ remains large for a much longer period, as the contraction becomes increasingly "weak."

Furthermore, according to the threshold derived in Question 8, as γ increases, the condition $\gamma \geq 1 - \frac{p_{\max}}{2p_{\min}}$ is more easily satisfied. This suggests that for long-horizon tasks, the **algorithmic contraction rate** almost always becomes the primary bottleneck for convergence, overshadowing the decay of stochastic noise. Intuitively, this reflects the difficulty of propagating reward information back through long chains of state-action pairs when future values are not heavily discounted.

Question 10: Uniform-Sampling

[1 Point]

Suppose a practitioner ensures a perfectly uniform exploration strategy such that every state-action pair is sampled with strictly equal probability ($p_{\min} = p_{\max}$). If they set $\gamma = 0.9$, which rate dictates the convergence speed, and what is the exact polynomial exponent of the decay?

Answer:

To determine the governing convergence rate, we evaluate both bounds provided by the theorem

under the given conditions.

For a perfectly uniform exploration strategy, the sampling probabilities are equal, which implies:

$$\bar{R} = \frac{p_{\min}}{p_{\max}} = \frac{p_{\max}}{p_{\max}} = 1$$

Given the discount factor $\gamma = 0.9$, we calculate the exponents for both candidate rates.

- Rate 1 (Algorithmic Contraction): The decay is $O(t^{-\bar{R}(1-\gamma)})$. Substituting the values:

$$\text{Exponent}_1 = \bar{R}(1 - \gamma) = 1 \cdot (1 - 0.9) = 0.1 \implies \text{Rate 1 is } O(t^{-0.1})$$

- Rate 2 (Stochastic Noise): The decay is $O\left(\sqrt{\frac{\log \log t}{t}}\right)$. Ignoring the slowly varying logarithmic term, the power-law exponent is:

$$\text{Exponent}_2 = 0.5 \implies \text{Rate 2 is approximately } O(t^{-0.5})$$

The overall convergence speed is dictated by the **slower** of the two rates, which corresponds to the smaller exponent. Since 0.1 is strictly smaller than 0.5, the algorithmic contraction rate is the dominant bottleneck.

Alternatively, using the threshold condition from Question 8:

$$\gamma \geq 1 - \frac{p_{\max}}{2p_{\min}} \implies 0.9 \geq 1 - \frac{1}{2} \implies 0.9 \geq 0.5$$

The condition holds, confirming that the **first rate** (the algorithmic contraction bound) dictates the speed. The exact polynomial exponent of the decay is **0.1**.